

Facultad de Ingeniería y Ciencias

Laboratorio de Arquitectura y organización de computadores

Laboratorio N^o [1] – Introducción a Vivado y a FPGA NEXYS 4 DDR

Integrantes:

Integrante 1 Mair Escobar

Integrante 2 Alonso Iturra

Integrante 3 Francisco Piñera

Sección: 03

Profesor Cátedra: Pablo Sánchez

Profesor Laboratorio: Jorge Tobar

Fecha de Entrega: 01/09/2025

Grupo: 6

Facultad de Ingeniería y Ciencias

Índice

1. Introducción	2
2. Desarrollo	2
2.1. Procedimiento Teórico	2
2.1.1. FPGA	2
2.1.2. VHDL	2
2.1.3. Funciones booleanas	3
2.2. Diseño, metodología y ejecución experimental	3
2.2.1. Configuración del entorno Vivado	3
2.2.2. Implementación del diseño VHDL	3
2.2.3. Pruebas funcionales en la FPGA	5
2.3. Análisis e interpretación de datos	8
2.4. Comparación de resultados con la teoría	9
2.4.1. NOT	9
2.4.2. AND	9
2.4.3. NAND	9
2.4.4. OR	10
2.4.5. NOR	10
2.4.6. XOR	10
2.4.7. NXOR	11
2.4.8. IF	11
3. Conclusiones	12

1. Introducción

Para el desarrollo de la experiencia fue necesario saber un par de conceptos relacionados a la computación, entre los que se encuentran las funciones booleanas, compuertas lógicas y conocimientos en FPGA, como a su vez sus entornos de desarrollo como el caso de Vivado. Se espera lograr una exitosa implementación de las funciones programadas en Vivado en la placa Nexys 4, y con ello comprobar si las compuertas lógicas se comportan de manera esperada al manejar los switches.

2. Desarrollo

2.1. Procedimiento Teórico

2.1.1. FPGA

Una FPGA o Field Programmable Array es un conjunto de circuitos integrados, su principal característica es que viene sin configurar, por lo que cada usuario lo debe hacer por si mismo, por lo que es muy adaptable a las necesidades del usuario, los FPGAs suelen tener diferentes bloques de procesamiento en su interior y diferentes conectores para poder configurar los diferentes bloques de lógica, pudiendo hacer operaciones complejas o compuertas lógicas, como en este caso, en adición esta es diferente a los microcontroladores como los Arduino por su capacidad de adaptabilidad a soluciones exigentes mientras que los microcontroladores están pensados para proyectos más pequeños y de menor consumo, otra diferencia notable es la forma en las que estos funcionan, mientras que si se envía código a un microcontrolador, ya viene listo para procesar, en cambio en una FPGA, es de suma importancia configurar antes el chip o chips que se vayan a utilizar.

2.1.2. VHDL

Este es un lenguaje descriptor de hardware, utilizado para describir formalmente y el diseño del hardware de circuitos electrónicos. Este lenguaje cuenta con 4 tipos de objetos:

- Constantes: Básicamente valores constantes en el código.
- Variables: Valores que pueden ir cambiando a medida de la ejecución.
- Señales: Representan las conexiones del hardware.
- Archivo: Estos son objetos que permiten la escritura y lectura de datos externos en simulaciones, pero no en la FPGA directamente.

Ademas de contar con operadores matemáticos también incluye operadores lógicos, en adición este es un lenguaje concurrente osea que se pueden ejecutar varias instrucciones al mismo tiempo para el caso de circuitos paralelos, se estructura en bloques, por lo que es posible escoger cual de todos los módulos se van a ocupar. Principalmente se ocupa en las simulaciones y en la síntesis del código.

2.1.3. Funciones booleanas

Las funciones booleanas fueron creadas por el matemático George Boole en el siglo XIX y estas tienen como valores de entradas 0 y 1 junto con el álgebra de Boole que define las operaciones y reglas para trabajar con el conjunto 0,1 son usadas en la electrónica para diseñar circuitos entre otras muchas funciones que tienen las funciones booleanas.

2.2. Diseño, metodología y ejecución experimental

2.2.1. Configuración del entorno Vivado

La configuración del programa abarca una gran cantidad de pasos preliminares antes de poder usarse. Sin extender mucho la explicación, El tipo de cosas que se necesitan seleccionar son:

- Tipo de proyecto: configurado como proyecto RTL
- Lenguaje objetivo y tipo de archivo: Para este caso se utilizara VHDL en ambos
- El dispositivo: Aquí de una larga lista es donde se debe seleccionar el equipo FPGA que se utilizara. Para este caso se utilizara el modelo específico XC7A100tcsg324-1. (XC7A100T es el modelo, csg324 indica que tiene 324 pines y el -1 indica el grado de velocidad).

2.2.2. Implementación del diseño VHDL

Este es uno de los pasos mas importantes ya que aquí se definen cuantas entradas y cuantas salidas podrán ser configuradas en la FPGA. Dependiendo de la cantidad, el usuario podrá definir si quiere trabajar con solo una parte de las combinaciones posibles en la FPGA o si en cambio busca utilizarlas todas.

La definición de las entradas y las salidas estarán dadas como entradas “Din” y salidas “Dout”, aunque esto solo son nombres descriptivos y no de carácter obligatorio. Para esta ocasión, dadas las necesidades de probar un gran numero de combinaciones en compuertas lógicas, se opto por utilizar 2 entradas de los switches y 8 salidas de los leds. Las entradas quedan definidas en representación de cuantos switches se planean utilizar mas el estado que puede adoptar cada switch (0 y 1), y las salidas quedan definidas en representación del número de diodos LED que se prenderán en función de como estén prendidos los dos switches. Es por ello que las salidas son muchas en comparación a las entradas.

Una vez que la configuración esta lista se despliega la interfaz del proyecto y con ello todas las opciones disponibles. A pesar de todo esto, la configuración aun no esta completa. Para completar la configuración se debe realizar un paso extra el cual consta de configurar los pines que utilizaran las entradas y salidas. Para configurar aquello es requerido crear un archivo de restricciones. Este archivo de restricciones contendrá una instrucción individual para cada entrada y salida, en la cual se le definirá que pin específico (dado por un código hexadecimal) utilizara en la placa.

```
set_property -dict { PACKAGE_PIN J15 IOSTANDARD LVCMOS33 } [get_ports { Din0 }];
set_property -dict { PACKAGE_PIN L16 IOSTANDARD LVCMOS33 } [get_ports { Din1 }];
set_property -dict { PACKAGE_PIN H17 IOSTANDARD LVCMOS33 } [get_ports { Dout1 }];
set_property -dict { PACKAGE_PIN K15 IOSTANDARD LVCMOS33 } [get_ports { Dout2 }];
set_property -dict { PACKAGE_PIN J13 IOSTANDARD LVCMOS33 } [get_ports { Dout3 }];
set_property -dict { PACKAGE_PIN N14 IOSTANDARD LVCMOS33 } [get_ports { Dout4 }];
set_property -dict { PACKAGE_PIN R18 IOSTANDARD LVCMOS33 } [get_ports { Dout5 }];
set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports { Dout6 }];
set_property -dict { PACKAGE_PIN U17 IOSTANDARD LVCMOS33 } [get_ports { Dout7 }];
set_property -dict { PACKAGE_PIN U16 IOSTANDARD LVCMOS33 } [get_ports { Dout8 }];
set_property -dict { PACKAGE_PIN V16 IOSTANDARD LVCMOS33 } [get_ports { Dout9 }];
set_property -dict { PACKAGE_PIN T15 IOSTANDARD LVCMOS33 } [get_ports { Dout10 }];
```

Figura 1: Restricciones de entradas y salidas

```
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity mux21 is
    Port ( Din1 : in STD_LOGIC;
          Din0 : in STD_LOGIC;
          Dout1 : out STD_LOGIC;
          Dout2 : out STD_LOGIC;
          Dout3 : out STD_LOGIC;
          Dout4 : out STD_LOGIC;
          Dout5 : out STD_LOGIC;
          Dout6 : out STD_LOGIC;
          Dout7 : out STD_LOGIC;
          Dout8 : out STD_LOGIC;
          Dout9 : out STD_LOGIC;
          Dout10 : out STD_LOGIC);
end mux21;

architecture Behavioral of mux21 is

begin

    Dout1 <= not Din0;
    Dout2 <= (Din0 and Din1);
    Dout3 <= not (Din0 and Din1);
    Dout4 <= (Din0 or Din1);
    Dout5 <= not (Din0 or Din1);
    Dout6 <= (Din0 and not Din1) or (Din1 and not Din0);
    Dout7 <= not ((Din0 and not Din1) or (Din1 and not Din0));
    Dout8 <= Din0;

end Behavioral;
```

Figura 2: Código VHDL

Ahora que la configuración esta completa, ya es posible programar las funciones VHDL para la placa. La programación del código se realiza dentro del archivo VHDL, acotado entre las secciones “begin” y “end behavioral”. El código completo utilizado para la programación de las compuertas lógicas es el siguiente:

2.2.3. Pruebas funcionales en la FPGA

A partir de este momento es donde se empieza a utilizar la FPGA. Para hacerla funcionar se debe conectar la placa mediante un cable y también conectarla desde el propio programa. una vez que se tiene la programación del VHDL lista, para pasarla del programa al dispositivo se debe generar el “Bitstream” y luego proceder a “programar dispositivo”. Aquello encenderá la placa obedeciendo las funciones programadas previamente. Si se desea cambiar sus funciones, es necesario re programar el fragmento de VHDL y repetir los dos pasos anteriores.

Acá una imagen para ver las compuertas lógicas simuladas con los leds y también de como están asignadas las entradas y las salidas en el dispositivo:

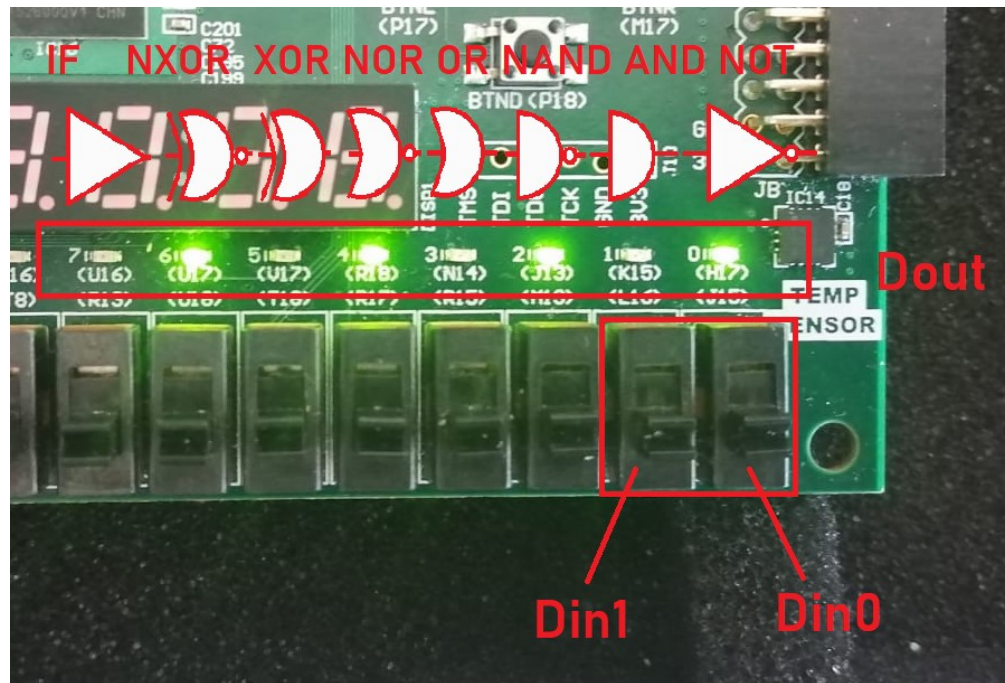


Figura 3: combinación 00

Para “testear” el funcionamiento del modelo VHDL hay que comprobar las combinaciones posibles de puertas lógicas. Dado que solo se cuenta con dos interruptores, la cantidad de estados posibles son 4.

Observación: Las columnas asociadas a las compuertas IF y NOT se relacionan con el estado del switch Din0.

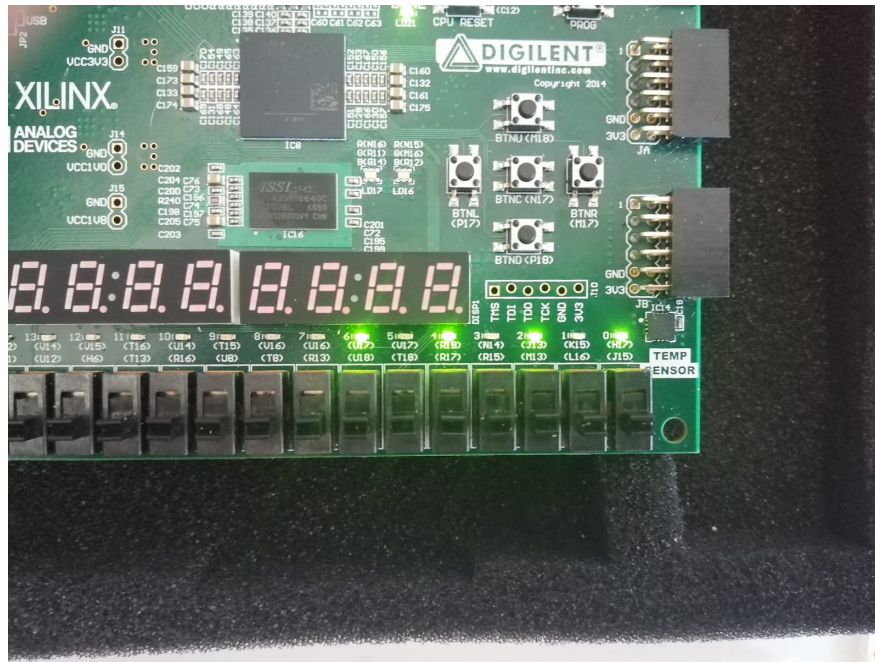


Figura 4: combinación 00 ($Din0 = 0$ y $Din1 = 0$)

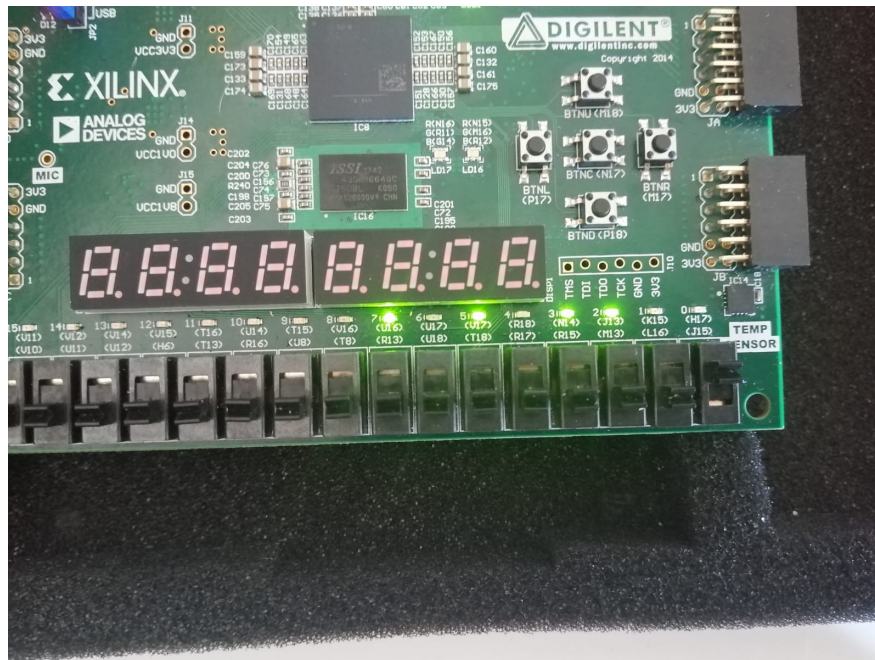


Figura 5: combinación 01 ($Din0 = 1$ y $Din1 = 0$)

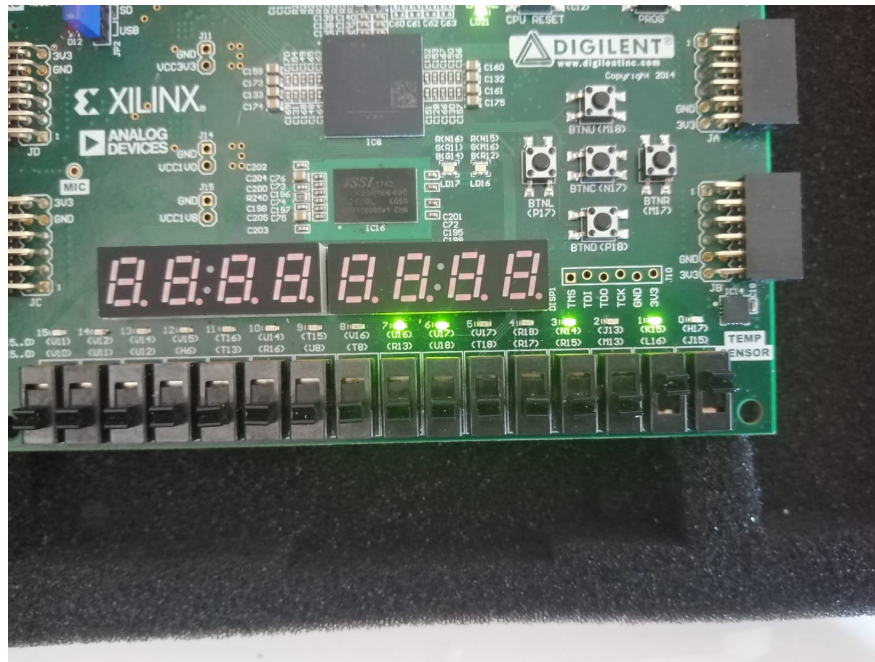


Figura 6: combinación 11 (Din0 = 1 y Din1 = 1)

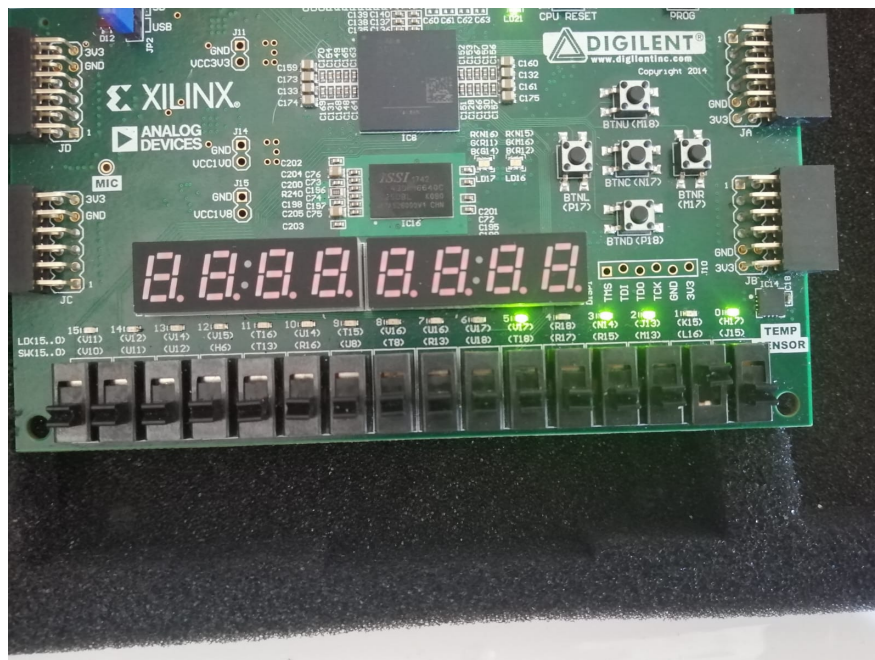


Figura 7: combinación 10 (Din0 = 0 y Din1 = 1)

Como se puede apreciar en la imagen se pueden ver todas las combinaciones de las dos variables y sus respectivas salidas.

Y finalmente se registraron los resultados obtenidos en la siguiente tabla:

Tabla 1: Tabla de resultados experimentales

Din0	Din1	Dout1 (NOT)	Dout2 (AND)	Dout3 (NAND)	Dout4 (OR)	Dout5 (NOR)	Dout6 (XOR)	Dout7 (NXOR)	Dout8 (IF)
0	0	1	0	1	0	1	0	1	0
0	1	1	0	1	1	0	1	0	0
1	0	0	0	1	1	0	1	0	1
1	1	0	1	0	1	0	0	1	1

2.3. Análisis e interpretación de datos

Una vez terminado el código se generó el bitstream para compilarlo, sin obtener errores. Luego de programar el dispositivo se hicieron pruebas sencillas para ver si se había cargado correctamente el código, como probar los switches configurados de manera individual y ver si los leds reaccionaban a estos cambios, además de comprobar el funcionamiento de compuertas sencillas como el IF, AND y OR.

Observando los tiempos de compilado y de síntesis, se determinó que en promedio duraban alrededor de 1-2 minutos. (Esa fue una de las principales razones por la cual se configuraron todas las compuertas en un solo programa).

En cuanto a simulaciones en Vivado, no fue posible realizar una simulación previa a la prueba real en la FPGA. Como resultado, para esta ocasión las simulaciones de esta experiencia fueron omitidas de manera excepcional.

2.4. Comparación de resultados con la teoría

Para corroborar que el procedimiento teórico y el experimental esta correcto es necesario comprobar las compuertas configuradas con sus respectivas tablas de verdad.

2.4.1. NOT

Para la compuerta NOT, solo es una variable de entrada, por lo que es más sencillo comprobar sus estados, basado en la figura 3 el led H17 es quien esta asignado a esa compuerta, revisando las figuras 3 y 4 cuando la entrada es 0, la salida es 1, y cuando la entrada es 1 la salida es 0, tal como se espera que funcione la compuerta NOT.

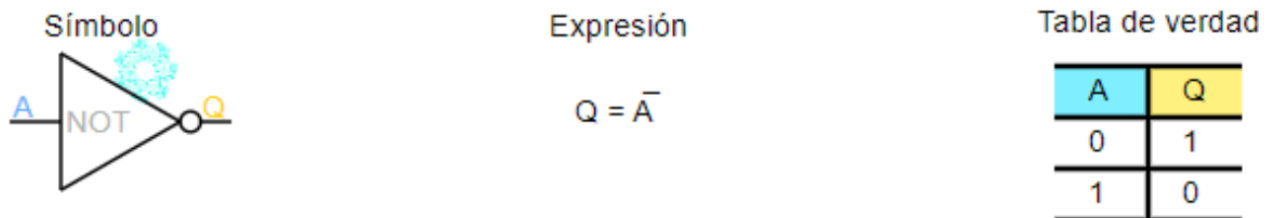


Figura 8: Compuerta NOT

2.4.2. AND

La compuerta AND requiere que ambos estén activados para poder funcionar, por lo que solo se activa con 1 y 1, revisando la asignación de entradas, le corresponde al led K15, revisando las combinaciones se tiene que con 1 y 1 esta encendido en cambio cualquier otra combinación este no se enciende.

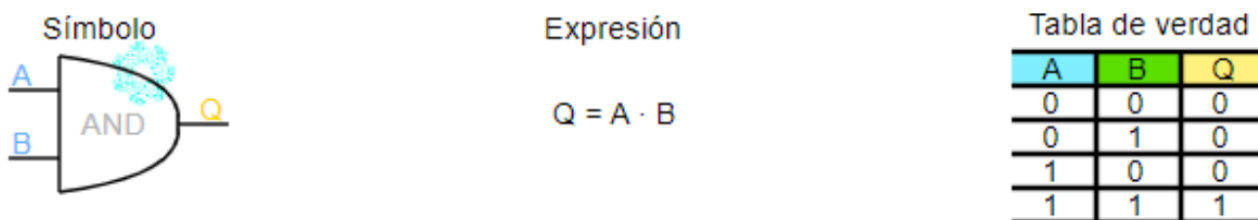


Figura 9: Compuerta AND

2.4.3. NAND

La compuerta NAND funciona como la negación de la compuerta AND por lo que no se activa con 1 y 1 pero el resto de combinaciones si, esta compuerta esta asignada al led J13, como se aprecia en las imágenes solamente en 1 y 1 el led no esta activado, comportándose como se esperaría una compuerta NAND.

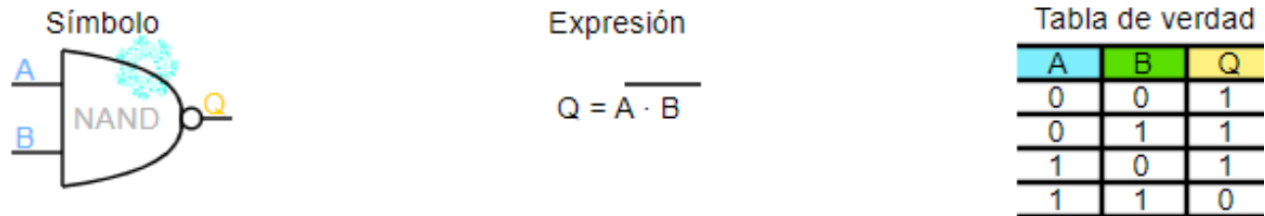


Figura 10: Compuerta NAND

2.4.4. OR

La compuerta OR para activarse requiere de una u otra o ambas activadas para encenderse, esta compuerta fue asignada al led N14, revisando sus entradas se puede comprobar que si funciona como una compuerta OR, al solo activarse con 01, 10 y 11.

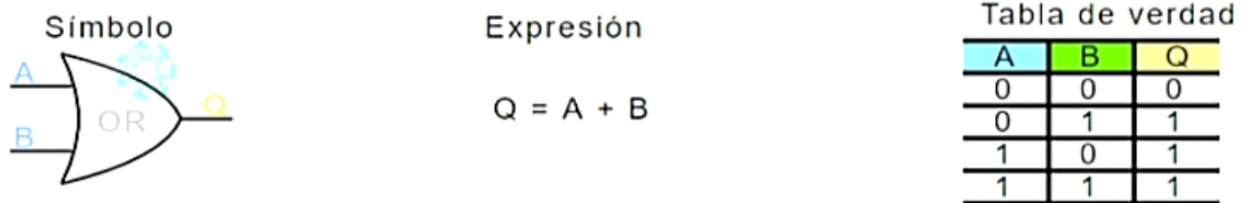


Figura 11: Compuerta OR

2.4.5. NOR

Esta compuerta funciona al contrario que el OR, solo se activa con la combinación 0 y 0, se led es el R18, revisando sus entradas, se puede apreciar que solo esta activado en 0 y 0, se comporta como se espera de una compuerta NOR.

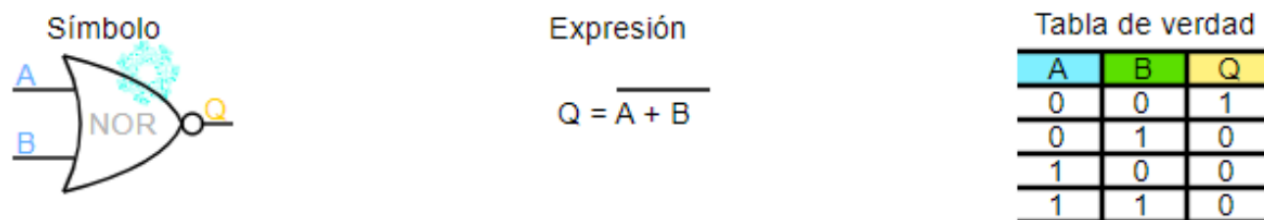
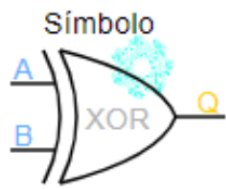


Figura 12: Compuerta NOR

2.4.6. XOR

El OR exclusivo es una compuerta que solo se activa solo si una de sus entradas es 1, si ambas son 0 o 1 esta no se activa, su led es el V17,comprobando las de resultados experimentales este solo se activa en 01 y 10.



Expresión

$$Q = A \oplus B$$

$$Q = A \cdot \bar{B} + \bar{A} \cdot B$$

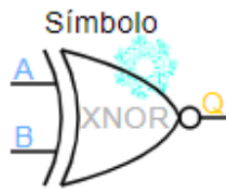
Tabla de verdad

A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

Figura 13: Compuerta XOR

2.4.7. NXOR

Esta compuerta es lo contrario al XOR, ya que solo se activa si ambas entradas son iguales, el led asignado es el U17, comparando la tabla de resultados este solo se activa con 00 y 11, por lo que tiene una reacción esperada para esa compuerta.



Expresión

$$Q = \overline{A \oplus B}$$

$$Q = A \cdot \bar{B} + \bar{A} \cdot B$$

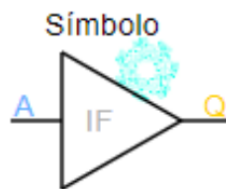
Tabla de verdad

A	B	Q
0	0	1
0	1	0
1	0	0
1	1	1

Figura 14: Compuerta NXOR

2.4.8. IF

La compuerta IF es la más sencilla, al variar de igual manera respecto al switch, su led es el U16, corroborando su tabla en resultados esta se comporta como un IF.



Expresión

$$Q = A$$

Tabla de verdad

A	Q
0	0
1	1

Figura 15: Compuerta IF

En resumen, se llego al resultado esperado, dado a que se logro obtener un 100 % de coincidencia de los outputs con su respectivas tablas de verdad.

3. Conclusiones

Una vez finalizada esta experiencia de laboratorio, se concluye que se realizaron las actividades satisfactoriamente, logrando los objetivos planteados de aprender en su ejecución a como diseñar, implementar y poner a prueba un modelo VHDL en una FPGA. En base a esto, se consiguió crear un modelo que permitiera comprobar el funcionamiento de las compuertas lógicas directamente en la FPGA moviendo interruptores.

Como resultado, se puede concluir que un buen entendimiento sobre el diseño y moldeamiento en FPGA es fundamental para comprender el funcionamiento de las compuertas lógicas en hardware físico, lo cual es muy importante para mas adelante poder tener una buena capacidad para diseñar el funcionamiento lógico de una pieza de hardware mas compleja.

Por otro lado, en evaluación de la experiencia con el entorno de Vivado, es concluyente reconocer que la calidad del programa en cuanto a funcionalidad y facilidad de uso es muy buena, siendo bastante sencillo poder crear, configurar y programar un modelo de VHDL el cual puede ser transmitido directamente a la FPGA en unos cuantos pasos. La interfaz es muy intuitiva incluso para un programa de este estilo y la mayoría de los procesos están casi automatizados (como por ejemplo la síntesis, la implementación y el bitstream). Es por ello que es destacable decir que como programa es muy confiable para el desarrollo en hardware.

Referencias

- [1] William Stallings, *Computer Organisation and Architecture 8e* by William Stallings, 2006.
- [2] M. Morris *Diseño digital*, 2003.
- [3] Javier Romero <https://www.geeknetic.es/FPGA/que-es-y-para-que-sirve>, 2021